

FIT2109 Computer Science Workshop

Week 9: Profiling & Performance

Yongqiang Tian

Department of Software Systems and Cybersecurity
Faculty of IT, Monash University

Acknowledgement

I wish to acknowledge the people of the Kulin Nations, on whose land we are gathered today. I pay my respects to their Elders, past and present.

"Premature optimization is the root of all evil."

— Donald Knuth, *Computer Programming as an Art*, 1974

- Week 8: you found and fixed the bug. Now: is the correct program *fast enough*?
- Make it work → make it correct → make it fast.
- Never optimise something you have not measured — you will almost always guess the wrong bottleneck.

Learning goals

By the end of this workshop, you should be able to:

- Explain the difference between benchmarking and profiling, and choose the right tool.
- Measure execution time reliably with `time` and `timeit`, accounting for variability.
- Use `cProfile` to identify which function consumes the most CPU time.
- Use `line_profiler` to pinpoint the specific line inside a hot function.
- Measure peak memory usage with `memory_profiler`.
- Apply the optimise → measure → verify loop without premature optimisation.

What makes a program feel slow to you?

Concept 1: Correctness comes before speed

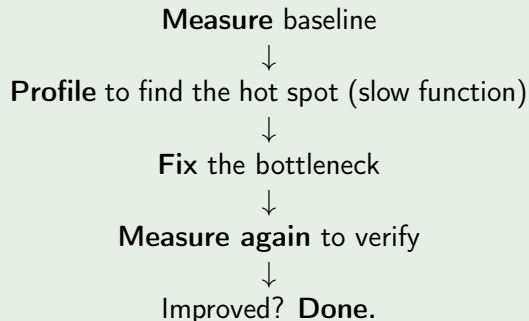
The full Knuth rule

"We should forget about small efficiencies, say about 97% of the time. *Yet we should not pass up our opportunities in that critical 3%.*"

What this means in practice

- 1 Write correct, readable code first.
- 2 Measure before touching anything.
- 3 Fix only what the profiler *confirms* is slow.

The performance workflow



Concept 2: Timing has more than one number

Three time components

- real** Wall-clock time you experience waiting.
- user** CPU time in your code.
- sys** CPU time in OS kernel calls.

Timing is noisy

A single run is unreliable: OS scheduling, CPU frequency scaling, and caching all add variance. Run 3–5 times; discard the first (*warm-up*: the first run is slower because CPU caches are cold) and use the median.

Basic usage

```
time python3 slow.py
# real  0m2.134s
# user  0m2.098s
# sys   0m0.036s
```

Modern alternative

hyperfine benchmarks shell commands with automatic statistical analysis (mean, min, max, stddev).

Concept 3: `timeit` reduces measurement noise

Why `timeit` over `time.time()`?

- Disables garbage collection during timing.
- Runs the statement many times automatically.
- Takes the **best** result across multiple trials.
- Handles warm-up implicitly.

Key flags

- `-n N` Iterations per trial.
- `-r N` Number of trials (keeps best).
- `-s STMT` Setup code (not timed).

Command-line

```
python3 -m timeit "sum(range(1000))"
```

```
python3 -m timeit \  
-s "data = list(range(1000))" \  
"sum(data)"
```

Choose the right tool

`timeit`: expressions. `time`: whole scripts.
Profiler: when you need to know *where* the time goes.

Demo 1: time and timeit in action

Live on Terminal — run `make setup WEEK=9` first

```
cd seminar/demo/week9/workspace/timing_demo
cat slow.py && cat fast.py      # review both files
time python3 slow.py          # string concat in a loop
time python3 fast.py          # join() approach -- faster
# micro-benchmark: generator vs list comprehension
python3 -m timeit \
    -s "data = list(range(500))" \
    "'-'.join(str(x) for x in data)"
python3 -m timeit \
    -s "data = list(range(500))" \
    "'-'.join([str(x) for x in data])"
```

You want to compare two Python expressions to see which is faster. Which tool should you reach for first?

- A `time python3 script.py` — wall-clock timing at the shell.
- B `python3 -m timeit "expression"` — repeated micro-benchmark.
- C `python3 -m cProfile script.py` — full CPU profiler.
- D `import time; t = time.time()` — manual timer in the script.

Concept 4: Timing says it is slow; profiling says where

Benchmarking — *how long?*

Measures total execution time. Tools: **time**, **timeit**, **hyperfine**. Use when comparing two implementations end-to-end.

Profiling — *where?*

Measures time per function or per line. Tools: **cProfile**, **line_profiler**. Use when the program is slow but you don't know why.

Analogy

Benchmarking is a stopwatch from start to finish.

Profiling is an X-ray — it reveals which part is doing all the work.

Common mistake

Benchmarking tells you the program is slow. Only profiling tells you *why* — and the bottleneck is almost never where you expect.

Concept 5: cProfile ranks function costs

Sample output (-s cumtime)

ncalls	tottime	percall	cumtime	percall	filename(func)
1	0.000	0.000	3.215	3.215	app.py(main)
1	0.001	0.001	3.215	3.215	app.py(process)
1	3.210	3.210	3.210	3.210	app.py(slow_op)
1	0.004	0.004	0.004	0.004	app.py(fast_op)

Column meanings

`ncalls` Times called.

`tottime` Time *inside* this function only.

`cumtime` Time *including* all sub-calls.

`percall` Average per call.

How to run

```
python3 -m cProfile -s cumtime app.py
```

Sort by cumtime first

`cumtime` reveals which function is *responsible* for the most time, even via sub-calls. Start there.

Demo 2: cProfile on a slow program

```
cd seminar/demo/week9/workspace/profile_demo

cat app.py                # review: several functions
python3 app.py            # takes a few seconds

python3 -m cProfile -s cumtime app.py
# which function has the highest cumtime?

python3 -m cProfile -s tottime app.py | head -15
# which function's own code is the hottest?
```

What to look for

The function with the highest `cumtime` is responsible for most of the runtime. The function with the highest `tottime` is where the CPU is actually burning cycles.

What flag do you pass to `python3 -m cProfile` to sort the output by cumulative time?

Concept 6: line_profiler finds the hot line

What it adds beyond cProfile

cProfile names the *function*. **line_profiler** names the *exact line inside that function* — the precise fix target.

Sample output (time in μ s)

Line	Hits	Time(us)	%Time	Contents
5	1001	234.0	2.1	for item in items:
6	1000	9876.0	88.7	result += str(item)
7	1000	1012.0	9.1	out.append(result)

How to use

```
pip install line_profiler

# Add @profile to the hot function
# kernprof injects it -- no import needed

kernprof -l -v hot.py
```

%Time is the key column

Line 6 consumes 88.7% of the function's time — that is the line to fix. Everything else is noise.

In cProfile output, a function shows `tottime = 0.001s` but `cumtime = 4.8s`. What does this tell you?

- A The function has a bug — `tottime` and `cumtime` should always match.
- B The function itself is fast, but it calls a slow sub-function — use `line_profiler` to drill into it.
- C The function was called only once, so the times should be equal.
- D The function is blocked on I/O, not CPU.

Demo 3: line_profiler on the hot function

```
cd seminar/demo/week9/workspace/lineprof_demo

cat hot.py      # @profile marks the slow function

kernprof -l -v hot.py
# output: per-line timing with %Time column
# -- which line dominates?

# fix the hot line, then re-verify with cProfile
python3 -m cProfile -s cumtime hot_fixed.py
```

The two-step investigation

Step 1: cProfile names the function. **Step 2:** line_profiler names the line. Together they give you a precise fix target.

After running `line_profiler` on a function you wrote, which line had the highest % Time — and were you surprised by the result?

Concept 7: memory_profiler tracks memory growth

Why memory matters

- Excessive allocation triggers garbage collection (GC) pauses.
- Loading a large file into a list when streaming suffices.
- Building intermediate structures held in RAM longer than needed.

How to use

```
pip install memory_profiler
```

```
# Add @profile; -m injects it (no import needed)
python3 -m memory_profiler leaky.py
```

Sample output

Line	Mem usage	Increment	Contents
5	48.0 MiB	48.0 MiB	def load_data(n):
7	125.4 MiB	77.4 MiB	big = list(range(n))
8	163.2 MiB	37.8 MiB	result = [x*2 for x in big]
9	163.2 MiB	0.0 MiB	return result

Increment is the key column

Line 7 allocates 77 MiB. If big is never reused after line 8, replace it with a generator to avoid the allocation.

Demo 4: memory_profiler and a memory fix

```
cd seminar/demo/week9/workspace/memprof_demo

cat leaky.py          # builds a large list unnecessarily
python3 -m memory_profiler leaky.py
# note: which line has the biggest Increment?

cat leaky_fixed.py    # uses a generator instead
python3 -m memory_profiler leaky_fixed.py
# note: peak memory is now much lower
```

Before and after

Replacing a large intermediate list with a generator can cut peak memory by 80%+ with no change to the final result — and often speeds things up too.

A function builds a list of 10 million numbers, filters it down to 10, and returns the 10. What is the most effective fix?

- A Use a faster CPU with more RAM.
- B Replace the list with a generator expression so only one item is held in memory at a time.
- C Use `line_profiler` to time each line of the function.
- D Store the intermediate list in a database instead of RAM.

Concept 8: Optimise only what you can verify

Three rules

- 1 **Measure first.** Have a baseline number before touching any code.
- 2 **One change at a time.** If you change two things, you cannot know which one helped.
- 3 **Verify the improvement.** Re-run the same benchmark. A gain of $< 10\%$ may be noise — check it.

Common Python bottlenecks

String concat Use `join()` or a list buffer.

List lookup Use a set or dict for membership.

Nested loops Seek vectorised alternatives.

Repeated I/O Cache results; read once.

When to stop

Stop when the program is *fast enough* for its use case, not when you can no longer find anything to optimise.

You fix a bottleneck and re-measure: runtime improves from 5.2s to 5.1s (about 2%). What is the right next step?

- A Keep optimising — even small gains compound over time.
- B Run the benchmark several more times: the 2% gain may be within normal timing noise.
- C Ship immediately — any improvement is worth deploying.
- D Revert the change — improvements under 10% are never worthwhile.

Activity: profile and fix `slow_data.py`

Solo or in pairs — 12 minutes

`workspace/activity/slow_data.py` has a bottleneck. Apply the full profiling workflow.

Workflow checklist

- 1 **Measure:** `time python3 slow_data.py` — how long does it take?
- 2 **Profile:** `python3 -m cProfile -s cumtime slow_data.py` — which function?
- 3 **Drill:** add `@profile` to that function; run `kernprof -l -v slow_data.py` — which line?
- 4 **Fix:** apply one targeted change.
- 5 **Verify:** `time python3 slow_data.py` again — a correct fix yields $\geq 5\times$ speedup.

Hint

The bottleneck is a membership test inside a loop. A set lookup is $O(1)$; a list lookup is $O(n)$.

Which step in the profiling workflow — measure, profile, drill, fix, verify — took longer than you expected, and why?

What should feel different after today?

You should now be able to

- Choose `time/timeit` vs a profiler based on what question you need to answer.
- Run `cProfile` and identify the bottleneck function from the output.
- Run `line_profiler` and identify the hot line inside that function.
- Spot unnecessary memory allocations with `memory_profiler`.
- Apply exactly one fix, re-measure, and decide whether to continue.

Tools at a glance

Question	Tool
How long overall?	<code>time</code>
Faster expression?	<code>timeit</code>
Which function?	<code>cProfile</code>
Which line?	<code>line_profiler</code>
Memory usage?	<code>memory_profiler</code>

Rank these steps in the order you would use them on an unfamiliar slow program:

- Measure the baseline with `time`
- Run `cProfile` to find the slow function
- Use `line_profiler` to pinpoint the slow line
- Fix one thing and re-measure

Answer individually

- ① You run `time python3 app.py` and see `real 5.2s, user 0.1s, sys 0.1s`. What is the likely bottleneck, and what would you investigate next?
- ② After `cProfile -s cumtime`, a function shows `tottime = 0.002s` and `cumtime = 4.8s`. What does this tell you about where to look next?
- ③ *Reflection*: Think of a program you have written that felt slow. If you had run `cProfile` on it, where do you think the time was really going — and would it have matched your intuition?

Name one profiling tool or technique from today that you will use the next time your code feels slow.

Coming up

- **Applied class:** profile and optimise a provided data-processing script end-to-end.
- **Week 10:** Data Wrangling — processing CSV, JSON, and text with shell tools and Python.

Resources

- docs.python.org/3/library/profile.html — cProfile documentation.
- github.com/pyutils/line_profiler — line_profiler and kernprof.
- github.com/pythonprofilers/memory_profiler — memory_profiler.

Questions?